



UCLINUX PORTING HOWTO

Monday 26th of July 2004, 04:30:56 pm. Posted in: **Embedded**. 8804 words.

Board specific configuration:

uclinux/Makefile

The toplevel Makefile is modified slightly to incorporate the changes in the building of the romfs image. The uClinux Make process proceeds as follows:

subdirs: User libraries (uclinux/lib/) are built, followed by selected user applications (uclinux/user).

romfs: The user applications are installed in the romfs directory (uclinux/romfs).

modules: Linux kernel modules are compiled.

modules_install: The compiled kernel modules are copied to the romfs directory (uclinux/romfs/lib/modules/).

image: The romfs image is generated at (uclinux/linux-2.4.x/romfs.o) from the romfs directory.

linux: The static components of the linux kernel are compiled to create the linux elf image (uclinux/linux-2.4.x/linux). The romfs image is included in the linux elf image, at the location specified by the linker script.

images: The final ARM7 binary is created.

```
...
...
...
ifeq (.config,$(wildcard .config))
include .config

all: subdirs romfs modules modules_install image linux images
else
all: config_error
endif
...
...
...
```

```

#

# normal make targets

#

.PHONY: romfs

romfs:

    for dir in $(DIRS) ; do [ ! -d $$dir ] || $(MAKEARCH) -C $$dir romfs || exit 1 ; done

    -find $(ROMFSDIR)/. -name CVS | xargs -r rm -rf

.PHONY: image

image:

    [ -d $(IMAGEDIR) ] || mkdir $(IMAGEDIR)

    $(MAKEARCH) -C $(VENDDIR) image

.PHONY: images

images:

    [ -d $(IMAGEDIR) ] || mkdir $(IMAGEDIR)

    $(MAKEARCH) -C $(VENDDIR) images

...

...

...

.PHONY: subdirs

subdirs:

    for dir in $(DIRS) ; do [ ! -d $$dir ] || $(MAKEARCH_KERNEL) -C $$dir || exit 1 ; done

dep:

    @if [ ! -f $(LINUXDIR)/.config ] ; then \

        echo "ERROR: you need to do a 'make config' first" ; \

        exit 1 ; \

    fi

    $(MAKEARCH_KERNEL) -C $(LINUXDIR) dep

...

...

...

```

uclinux/vendors/ACME/SOHO-B0X/Makefile

This is the vendor specific Makefile used to build the rom filesystem structure, generate the romfs image and create the linux binary.

romfs: The directory structure for the rom filesystem is created here. ROMFS_DIRS specifies the directories that are to be created, and DEVICES specifies the device files for various device drivers. The device files are of the format: DEVICE_NAME,character/block device,major number,minor number. Additionally a number of symbolic links are created and essential files copied to their respective locations in the romfs directory.

image: The romfs binary is created using the utilities genromfs and arm-elf-ld. The three commented commands concatenate the romfs binary at the end of the linux binary instead of within it.

images: The linux binary is created using arm-elf-ld and copied to the tftp download location.

```
#
#      Makefile -- Build instructions for ACME/SOHO-B0X
#
ROMFSIMG = $(IMAGEDIR)/romfs.img
IMAGE = $(IMAGEDIR)/image.bin
ELFIMAGE = $(IMAGEDIR)/image.elf

ROMFS_DIRS = bin dev etc home lib mnt proc usr var

DEVICES = \
    tty,c,5,0      console,c,5,1      cua0,c,5,64 \
    cua1,c,5,65 \
    \
    mem,c,1,1      kmem,c,1,2      null,c,1,3 \
    zero,c,1,5     random,c,1,8     urandom,c,1,9 \
    \
    ram0,b,1,0     ram1,b,1,1 \
    \
    ptyp0,c,2,0    ptyp1,c,2,1      ptyp2,c,2,2 \
```

```

ptyp3,c,2,3      ptyp4,c,2,4      ptyp5,c,2,5 \
ptyp6,c,2,6      ptyp7,c,2,7      ptyp8,c,2,8 \
ptyp9,c,2,9      ptypa,c,2,10     ptypb,c,2,11 \
ptypc,c,2,12     ptypd,c,2,13     ptype,c,2,14 \
ptypf,c,2,15 \
\
mtd0,c,90,0      mtd1,c,90,2      mtd2,c,90,4 \
mtd3,c,90,6      mtd4,c,90,8      mtd5,c,90,10 \
mtd6,c,90,12     mtd7,c,90,14 \
\
mtdblock0,b,31,0 mtdblock1,b,31,1 mtdblock2,b,31,2 \
mtdblock3,b,31,3 mtdblock4,b,31,4 mtdblock5,b,31,5 \
mtdblock6,b,31,6 mtdblock7,b,31,7 \
\
rom0,b,31,0      rom1,b,31,1      rom2,b,31,2 \
rom3,b,31,3      rom4,b,31,4      rom5,b,31,5 \
rom6,b,31,6      rom7,b,31,7      rom8,b,31,8 \
rom9,b,31,9 \
\
tty0,c,4,0      tty1,c,4,1      tty2,c,4,2 \
tty3,c,4,3      ttyS0,c,4,64     ttyS1,c,4,65 \
\
ttyp0,c,3,0      ttyp1,c,3,1      ttyp2,c,3,2 \
ttyp3,c,3,3      ttyp4,c,3,4      ttyp5,c,3,5 \
ttyp6,c,3,6      ttyp7,c,3,7      ttyp8,c,3,8 \
ttyp9,c,3,9      ttypa,c,3,10     ttypb,c,3,11 \
ttypc,c,3,12     ttypd,c,3,13     ttype,c,3,14 \
ttypf,c,3,15

```

clean:

romfs:

```

[ -d $(ROMFSDIR)/$$i ] || mkdir -p $(ROMFSDIR)

for i in $(ROMFS_DIRS); do \

```

```

        [ -d $(ROMFSDIR)/$$i ] || mkdir -p $(ROMFSDIR)/$$i; \

done

for i in $(DEVICES); do \

    touch $(ROMFSDIR)/dev/@@$$i; \

done

$(ROMFSINST) -s /var/tmp /tmp

$(ROMFSINST) -s /bin /sbin

$(ROMFSINST) /etc/rc

$(ROMFSINST) /etc/inittab

$(ROMFSINST) ../../Generic/romfs/etc/services /etc/services

case "$(LINUXDIR)" in \

*2.4.*) ;; \

*) echo "ttyS0:linux:/bin/sh" >> $(ROMFSDIR)/etc/inittab ;; \

esac

$(ROMFSINST) /etc/motd

$(ROMFSINST) /etc/passwd

echo "$(VERSIONSTR) -- " `date` > $(ROMFSDIR)/etc/version

image:

[ -d $(IMAGEDIR) ] || mkdir -p $(IMAGEDIR)

genromfs -v -V "ROMdisk" -f $(ROMFSIMG) -d $(ROMFSDIR)

arm-elf-ld -r -o $(ROOTDIR)/$(LINUXDIR)/romfs.o -b binary $(ROMFSIMG)

# $(CROSS_COMPILE)objcopy -O binary --remove-section=.romvec \

#         --remove-section=.text --remove-section=.ramvec \

#         --remove-section=.init \

#         --remove-section=.bss --remove-section=.eram \

#         $(ROOTDIR)/$(LINUXDIR)/linux $(IMAGEDIR)/linux.data

# $(CROSS_COMPILE)objcopy -O binary --remove-section=.ramvec \

#         --remove-section=.bss --remove-section=.data \

#         --remove-section=.eram \

#         --set-section-flags=.romvec=CONTENTS,ALLOC,LOAD,READONLY,CODE \

#         $(ROOTDIR)/$(LINUXDIR)/linux $(IMAGEDIR)/linux.text

# cat $(IMAGEDIR)/linux.text $(IMAGEDIR)/linux.data $(ROMFSIMG) > $(IMAGE)

```

```
images:

    arm-elf-objcopy -O binary -R .note -R .comment -S $(ROOTDIR)/$(LINUXDIR)/linux
$(IMAGE)

    cp $(IMAGE) /tftpboot/linux_bootram.bin
```

uclinux/vendors/ACME/SOHO-BOX/rc

The resource configuration script is executed upon system startup. The expand utility creates a read-write RAM filesystem and mounts it on /var. The network can be setup using the dynamic host configuration protocol. Finally, the system startup screen is printed.

```
hostname SOHO-BOX

/bin/expand /etc/ramfs.img /dev/ram0

mount -t ext2 /dev/ram0 /var

mkdir /var/config

mkdir /var/tmp

mkdir /var/lock

mount -t proc proc /proc

#ifconfig lo 127.0.0.1

#route add -net 127.0.0.0 netmask 255.255.255.0 lo

#ifconfig eth0 10.151.10.12 netmask 255.255.0.0

#route add -net 10.151.10.12 netmask 255.255.0.0 eth0

#dhcpcd -d -p -a eth0 &

cat /etc/motd
```

uclinux/vendors/ACME/SOHO-BOX/motd

Contents of this file are printed upon system startup.

```
Welcome to

      ____  _
     / __ \| | | |
    / /_ \| | | | _ _ _ _ _
   / ___ \| | | | _ \| | | \ /
  /_/   \| | | | | | | | | | \
 /_/   \| | | | | | | | | | \
```

```
|  _  \  _  |  |  |  |  |  \  _  |  \  \  /
|  |
|  |
```

For further information check:

<http://www.uclinux.org/>

uclinux/vendors/ACME/SOHO-B0X/inittab

Initialization script.

```
#inet:unknown:/bin/inetd
#boa:unknown:/bin/boa
```

uclinux/vendors/ACME/SOHO-B0X/passwd

User information for authenticated logins if required.

```
root ab6TRGT20sY26
```

uclinux/vendors/ACME/SOHO-B0X/config.arch

Architecture configuration options for the build process.

```
.EXPORT_ALL_VARIABLES:
#####
#
# Vendor specific settings
#
CONSOLE_BAUD_RATE = 115200

#####

CPUFLAGS :=
VENDOR_CFLAGS :=
DISABLE_XIP := 1
DISABLE_MOVE_RODATA := 1 # until we get a toolchain release
```

```
# LOPT := -O2 # library is alway 02

# UOPT := -Os # user apps are always Os

#####

include $(ROOTDIR)/vendors/config/armnommu/config.arch

#####
```

uclinux/vendors/ACME/SOHO-B0X/config.linux-2.4.x

Factory default configuration for linux; copied from uclinux/linux-2.4.x/.config.

uclinux/vendors/ACME/SOHO-B0X/config.uClibc

Factory default configuration for the uClibc library; copied from uclinux/uClibc/.config.

uclinux/vendors/ACME/SOHO-B0X/config.vendor-2.4.x

Factory default configuration for the selection of user applications; copied from uclinux/config/.config.

Jump to page: 1 2 3 4 5 6



Comments (1) |



Permalink